

Title & Overview

Mobile and Object-Oriented

Testing Specializations

A Step-by-Step Lecture Guide · Simple English · Every Topic Explained

1. Mobile Testing Challenges

2. Introduction to Appium

3. OO Inheritance Testing

4. OO Polymorphism Testing

Software Testing Course | Lecture Notes

Mobile Testing Basics

PART 1 — MOBILE TESTING

What is Mobile Testing?

DEFINITION

Mobile Testing means checking that an app works correctly on mobile phones and tablets.

We test how the app looks, how it behaves, and whether it crashes or gives wrong results.

Why Do We Need Mobile Testing?

Because mobile devices are very different from computers.

Every phone has a different screen size, different speed, and different software version.

An app that looks good on one phone may look broken on another phone.

People use phones everywhere — while walking, in cars, with bad network — so apps must work in all conditions.

REAL LIFE EXAMPLE

Imagine you build a banking app. You test it on your own phone and it works.

But a customer with an older Android phone sees a broken screen.

This is why we must test on many different devices — not just one.

Android Fragmentation

PART 1 — MOBILE TESTING

Android Fragmentation — The Biggest Challenge

DEFINITION

Fragmentation means that Android runs on thousands of different phones made by different companies.

Each company can change Android in its own way. So the same app can look and behave differently on each phone.

What Makes Android So Difficult to Test?

24,000+

Android Device Models

Samsung, Xiaomi, OnePlus, Huawei — all different!

8+

Active Android Versions

Many users never update their phone's software

4" – 7"

Range of Screen Sizes

Small screen vs big screen — layout must fit both

1,300+

Phone Manufacturers

Each brand modifies Android their own way

iOS Challenges & Comparison

PART 1 — MOBILE TESTING

iOS Testing Challenges + Android vs iOS Comparison

iOS — Fewer Devices, But Still Tricky

Apple makes all iPhones, so there are fewer device types.

But iOS still has testing challenges:

Many users still run old iOS versions (15, 16, 17 at the same time)

iPhone SE vs iPhone 15 Pro Max — very different sizes

You can only test iOS apps on a Mac computer (not Windows)

Apple has very strict App Store rules — bugs must be found before submitting

Android vs iOS: Quick Comparison

KEY LESSON

You cannot test only on YOUR phone. Real users have many different phones, screen sizes, and old software versions.

Always test on a mix of popular devices — both old and new.

Other Mobile Testing Challenges

PART 1 — MOBILE TESTING

Other Important Mobile Testing Challenges

Besides fragmentation, mobile testers must also think about these real-world situations that desktop testers almost never deal with:

■ Network Speed

Definition: The speed of the internet connection.

Challenge: Apps must work on fast 5G, slow 3G, and even when there is no internet at all.

■ Battery & Performance

Definition: How much power and memory the app uses.

Challenge: A heavy app drains the battery fast and becomes slow. Users will delete it.

■ Interruptions

Definition: Things that stop the app while it is running.

Challenge: A phone call, a text message, or a pop-up can pause your app. The app must recover correctly.

■ Permissions & Privacy

Definition: Access the app asks for (camera, location, contacts).

Challenge: Test what happens when the user says NO to a permission request.

■ Touch & Gestures

Definition: How the user controls the app with their fingers.

Challenge: Swipe, pinch, zoom, and long-press are hard to automate. Must be tested carefully.

■ Screen Orientation

Definition: Portrait (tall) mode vs Landscape (wide) mode.

Challenge: When the user rotates the phone, the app layout must look correct in both positions.

Introduction to Appium

PART 2 — INTRODUCTION TO APPIUM

What is Appium?

DEFINITION

Appium is a free, open-source tool used to automatically test mobile apps.

Instead of clicking the app by hand every time, you write a test script (a small program) that clicks and checks the app FOR you, automatically.

Why Do Testers Use Appium?

It is FREE — no need to pay for a licence.

One test works for BOTH Android AND iOS apps.

You do NOT need to change the app code to use it.

You can write tests in any language: Java, Python, JavaScript, Ruby.

It works with real phones AND emulators (fake phones).

Quick Facts

Created in:

2012

Maintained by:

OpenJS Foundation

Licence:

Apache 2.0 — completely free

Works on:

Android + iOS + Windows Apps

Protocol used:

WebDriver (same as Selenium)

SIMPLE ANALOGY

Think of Appium like a ROBOT HAND. Teach the robot which buttons to tap and what to check.

Then it does the job for you — again and again — perfectly.

How Appium Works

PART 2 — INTRODUCTION TO APPIUM

How Appium Works — Step by Step

Appium has three main parts that work together. Think of it like ordering food at a restaurant:

STEP 1

Your Test Script

(The Customer)

This is the code YOU write.

Example: "Tap the Login button" or

"Type 'hello' into the text box".

It sends instructions to Appium.

→

STEP 2

Appium Server

(The Waiter)

Appium Server receives your instructions and translates them.

It runs on your computer.

Start it by typing: appium

→

STEP 3

Mobile Device

(The Kitchen)

The real phone or emulator that runs the app.

Appium uses a "driver" to talk to Android or iOS directly.

EXPLANATION — WebDriver Protocol

Your test script talks to Appium using a standard called the WebDriver Protocol.

This is the SAME standard that Selenium uses for web testing.

That is why people who know Selenium can learn Appium quickly.

Appium Setup & Test Execution

PART 2 — INTRODUCTION TO APPIUM

How to Set Up Appium — Step by Step Guide

Follow these 5 steps in order. Do not skip any step.

1

Install Node.js

Appium needs Node.js to run. Go to nodejs.org and download it. After installing, open Terminal and type: `node --version` — you should see a version number.

2

Install Appium

In Terminal, type this command: `npm install -g appium`

This downloads Appium onto your computer. The `-g` means 'install for everyone on this computer'.

3

Install the Right Driver

For Android testing, type: `appium driver install uiautomator2`

For iOS testing, type: `appium driver install xcuitest`

Drivers are what let Appium talk to each platform.

4

Install Android Studio (for Android) or Xcode (for iOS)

Android: Download Android Studio and set the `ANDROID_HOME` path in your system settings.

iOS: Download Xcode from the Mac App Store. Note: iOS testing only works on Mac computers.

5

Start Appium and Run Your First Test

In Terminal, just type: `appium`

Appium starts and waits for test commands on port 4723. Now you are ready to write and run tests!

PART 2 — INTRODUCTION TO APPIUM

Writing a Test + Real Device vs Emulator

Example: Simple Appium Test (Python)

Step 1: Import Appium

```
from appium import webdriver
```

Step 2: Set Device Settings

```
options = AndroidOptions()
options.app = "/path/app.apk"
options.device_name = "MyPhone"
```

Step 3: Connect to Appium

```
driver = webdriver.Remote(
    "http://localhost:4723",
    options=options)
```

Step 4: Find and Tap a Button

```
btn = driver.find_element(
    "id", "com.app:id/login")
btn.click()
```

Step 5: Close the test

```
driver.quit()
```

Real Device vs Emulator

Real Device (Actual Phone)

■ Pros: Most accurate · Tests real hardware · Tests real network

■ Cons: Expensive · Hard to manage many phones

Emulator (Fake Phone on PC)

■ Pros: Free · Easy to set up · Can run many at once

■ Cons: Not 100% same as real phone · Cannot test camera/GPS

BEST PRACTICE

Use emulators for quick, daily testing during development.

Use real phones for the final test just before releasing the app to users.

■ Best Practice: Use emulators for quick daily testing. Use real phones for final checks before release.

OOP Concepts Review

PART 3 — TESTING OO SYSTEMS

Object-Oriented Programming (OOP) — A Quick Review

DEFINITION — What is OOP?

Object-Oriented Programming (OOP) is a way of writing code using "objects".

An object is like a real-world thing. For example: a "Car" object has properties (colour, speed) and actions (drive, stop).

Code is grouped into Classes (blueprints) and Objects (specific versions of those blueprints).

The 4 Main Ideas in OOP (You Need to Know These):

■ Encapsulation

Hiding the inside details of an object.

Example: You use a TV remote — you press ON and it works. You do not need to know how the TV works inside.

■ Inheritance

A child class gets everything from its parent class.

Example: Animal has eat() and sleep(). Dog is a child of Animal — so Dog also has eat() and sleep() automatically.

■ Polymorphism

The same method name works differently for different objects.

Example: shape.draw() draws a circle for a Circle object, but draws a square for a Square object.

■ Abstraction

Showing only what is needed and hiding everything else.

Example: When you drive a car, you just turn the wheel. You do not see the engine working.

Inheritance & Polymorphism Testing

PART 3 — TESTING OO SYSTEMS

Testing Inheritance — Problems & How to Handle Them

DEFINITION — What is Inheritance?

Inheritance is when a child class automatically gets all the code (methods and properties) from a parent class.

Child classes can also ADD new features and CHANGE (override) existing ones.

Common Problems (Pitfalls) When Testing Inheritance:

1

Changing the Parent Breaks ALL Children

When you change the parent class, ALL child classes that use its code are affected — even if you never touched them.

Example: Change `eat()` in `Animal` → must re-test `Dog` and `Cat` too.

2

Deep Inheritance Chains are Hard to Trace

Many levels (`A → B → C → D`) means a bug in `A` can break `D` in a surprising way.

Example: `Vehicle → Car → ElectricCar → TeslaModel3`. A tiny change in `Vehicle` can crash `TeslaModel3`.

3

Method Overriding Can Cause Surprises

A child class can replace (override) a parent method. If the child version has a bug, it breaks all code that calls that method.

Example: `Dog.speak()` overrides `Animal.speak()` but returns null instead of 'Woof' — this crashes the whole app.

STRATEGY: Test each class alone first. After changing a parent, re-test ALL child classes. Use mock objects to isolate each class.

PART 3 — TESTING OO SYSTEMS

Testing Polymorphism — Problems & Strategies

DEFINITION — What is Polymorphism?

Polymorphism means the same method name can do DIFFERENT things depending on WHICH object is calling it.

Example: `shape.area()` gives 78.5 if `shape` is a `Circle`, but gives 25 if `shape` is a `Square`.

Common Testing Pitfalls:

■■ Pitfall 1 — Testing Only One Type

Testers test `Circle.area()` and assume `Square.area()` also works. WRONG! Each class must be tested separately — they each behave differently.

■■ Pitfall 2 — Late Binding Bugs

The computer picks WHICH method to run only at runtime. A bug may only appear when a specific object is used — hard to find early.

■■ Pitfall 3 — Missing Method in Child Class

If a developer forgets to add a required method in a child class, the app CRASHES at runtime — no compile-time warning.

How to Test Polymorphism Correctly:

1. Test EVERY Class Separately

Do not assume that if one class works, all classes work. Write a separate test for each class that uses the polymorphic method.

2. Use Unit Tests with Mock Objects

Replace real dependencies with "mock" (fake) objects. This way you can test each class alone without needing other classes to work.

3. Test Runtime Behaviour

Write tests that run the app with different object types and check that each one gives the correct result — not just the first one you think of.

4. Integration Testing After Unit Tests

Once every class passes its own test, test them all working TOGETHER. Polymorphism bugs often only appear when multiple classes interact.

GOLDEN RULE: Always test EVERY class that uses a polymorphic method. NEVER assume 'if one works, all work'.